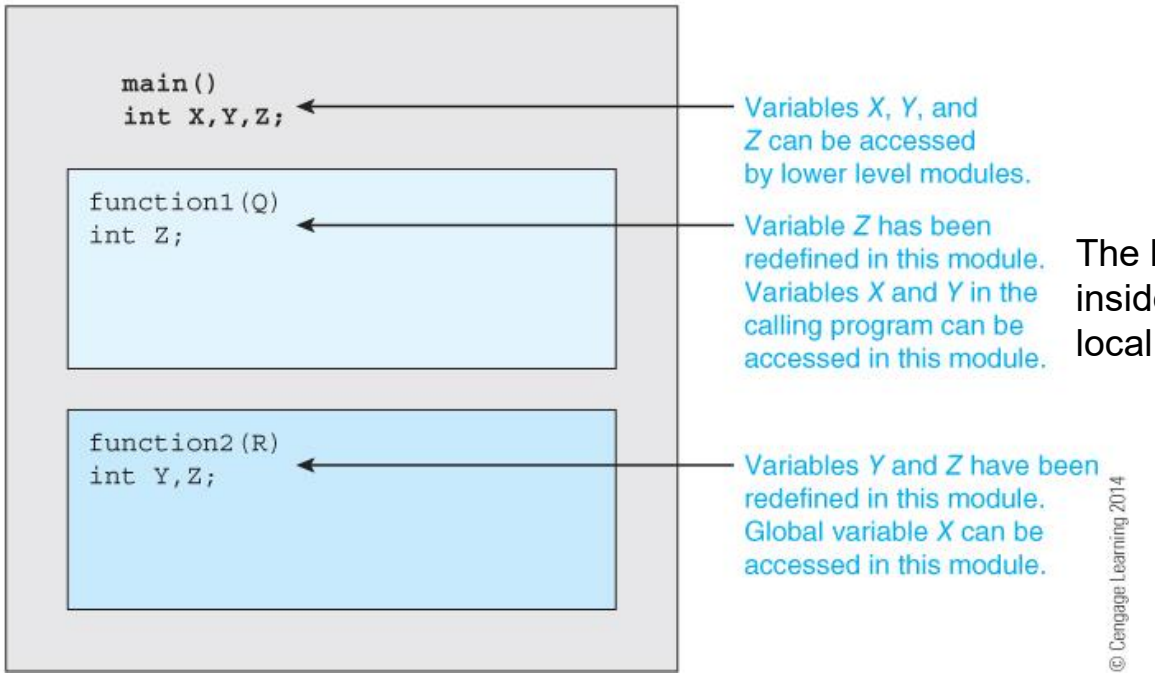
3.7 Stack, parameter passing and local variables

Stack frame

- In the subroutine section, we learned how to:
  - pass parameter in a subroutine
  - return a value in a register
  - preserve registers on a stack
- The mechanism would not be sufficient if:
  - we need more parameters than can be supported by built-in registers
  - we need to set aside local variables within a subroutine
- *Stack frame* is a more sophisticated mechanism for passing parameters, handling local variables and preserving registers.
- Some concepts before introducing the stack frame …..

Local and global variables

FIGURE 4.1 The concept of scope

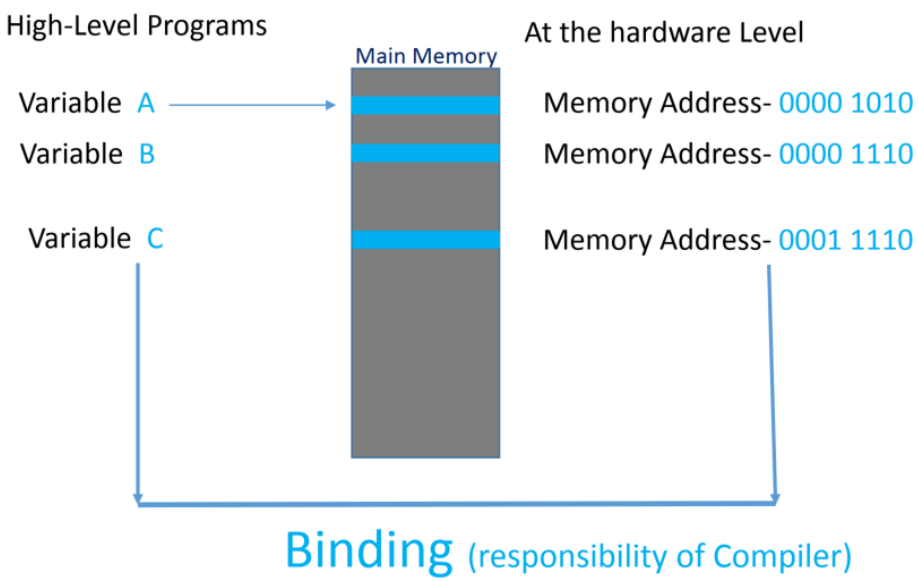


© Cengage Learning 2014

The local variable Z is only accessible inside function1. The scope of the local variable Z is within function1.

Binding

- Binding: Associating a variable with its storage location
- Static binding:
  - this association is made at compile time.
- Dynamic binding:
  - this association is made at run-time.



Recursion

- Recursive function: a function calling itself.

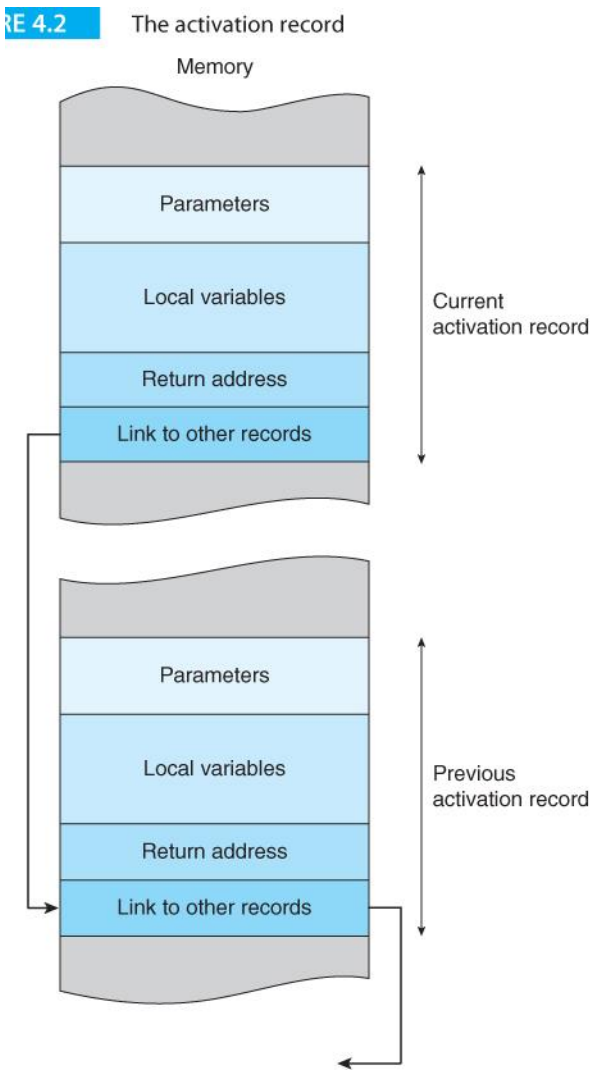
```
int Factorial(int n)
{
  if(n == 1)
    return 1;
  else
    return n * Factorial(n-1);
}
```

$$N! = N \times (N-1)!$$

- The local variable `n` exists at different levels of the call.
- Static binding does not permit recursion.
- We need dynamic data storage discussed below.

The stack and dynamic data storage

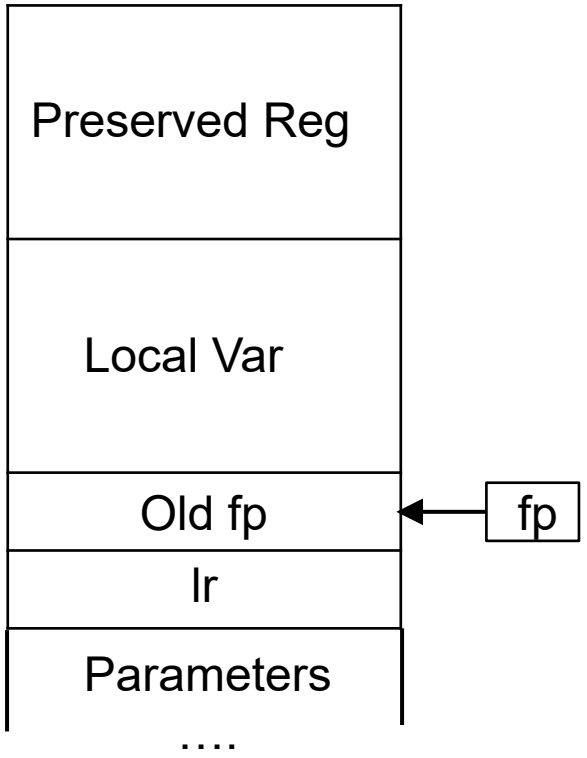
- When a subroutine is called, it is said to be *activated*.
- Each call is associated with an *activation record* (also called as *stack frame*)
- The activation record is the subroutine's view of the world.
- Recursion requires dynamic data storage because storage must be allocated at *runtime.*
- Executing a return from a subroutine *deallocates* or frees the storage taken up by the record.



Stack frame in subroutine calling

The stack frame involves three major registers:

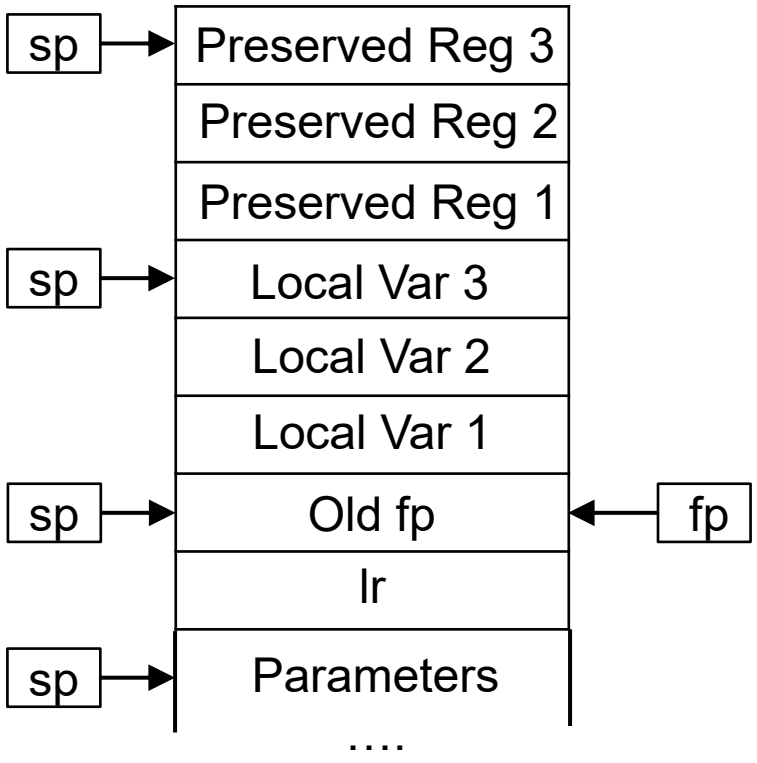
1. Stack pointer (sp) contains the address (points to) the top of stack.
2. Link register (lr) keeps track of the program line to return to after the subroutine finishes.
3. Frame pointer (fp) provides a reference point in the current activation record for users to locate parameters, local variables, saved link registers and preserved registers.



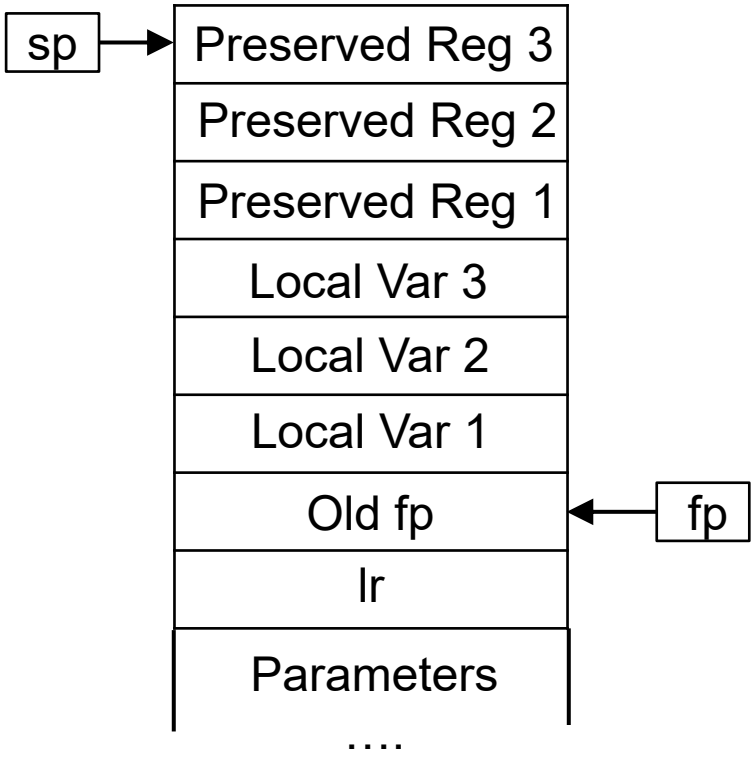
Stack frame prologue

The prologue of a subroutine consists of the following steps:

1. Push the fp and lr onto the stack
   - fp is being preserved
   - The subroutine is called by the Branch Link (bl) instruction, which stores the address of the instruction immediately following the subroutine call into lr.
2. Save the current sp to the fp.
3. Allocate local variable storage space on the stack.
4. Push register to preserve onto the stack



After prologue



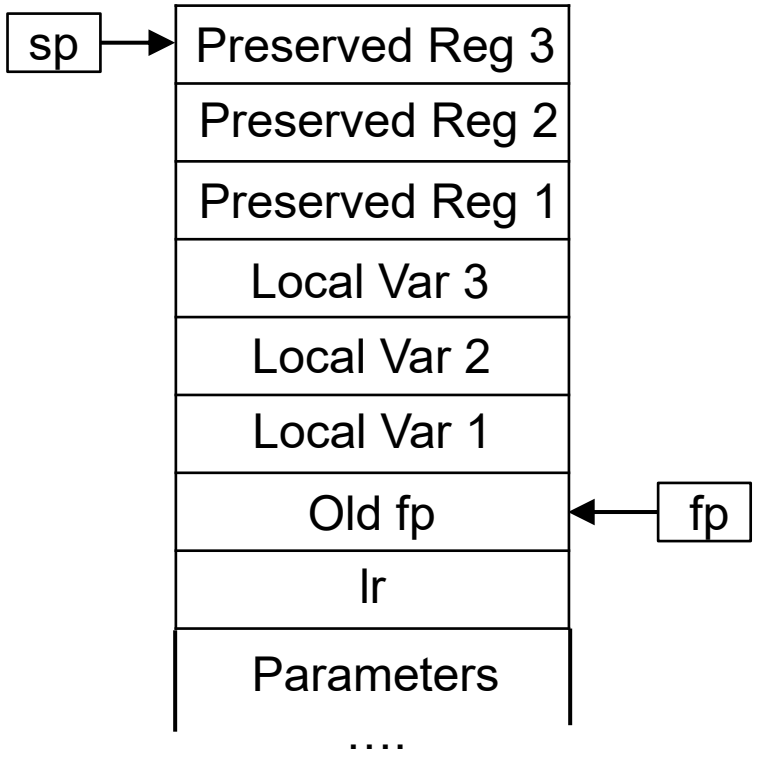
ARM code implementing prologue

```
Sub:
stmfd sp!, {fp, lr}
//Step 1


mov fp, sp //Step 2


sub sp, sp, #4 //Step 3:
assuming only one local
variable

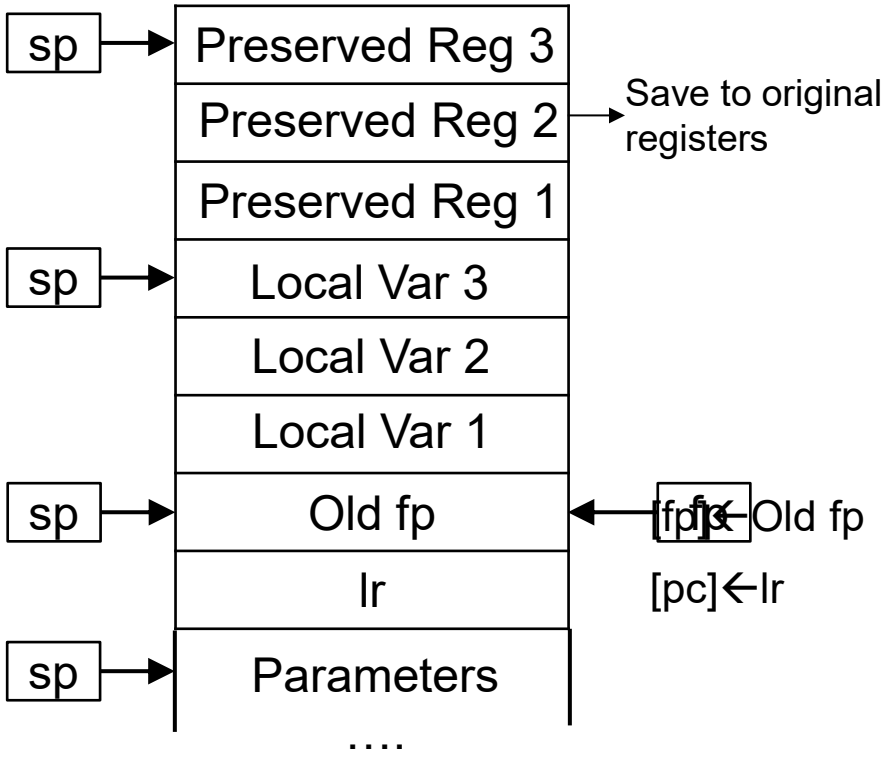

stmfd sp!, {r1-r3} //
Step 4: assuming r1 to r3
are used in the
subroutine
```



Stack frame epilogue

The epilogue of a subroutine consists of the following steps:

1. Pop preserved registers from the stack.
2. Deallocate local variable storage from the stack
3. Pop fp and pc from the stack
   - the pc now contains the saved lr → Execution resumes at the line after the subroutine call



ARM code implementing prologue

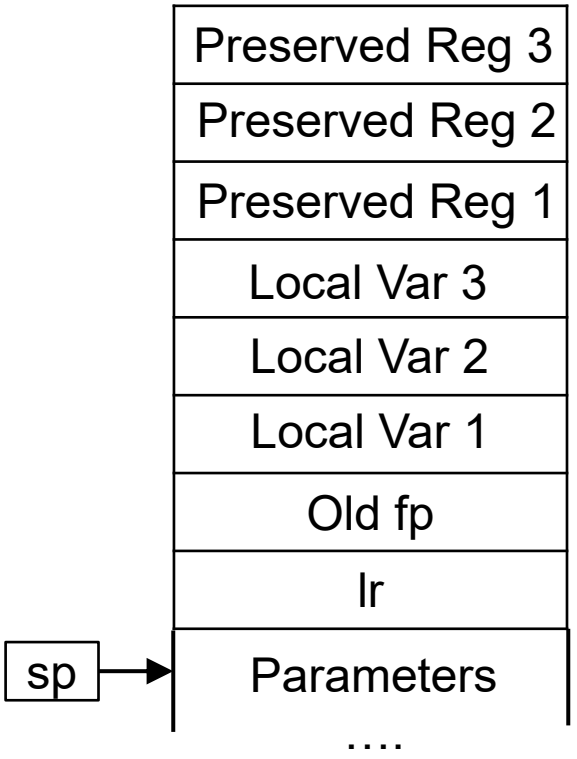
```
ldmfd sp!, {r1-r3} // Step 1

add sp, sp, #4 // Step 2

ldmfd sp!, {fp, pc} // Step 3
```

State after epilogue

- The state of memory, except for the returned values in `r0` and updated parameter values, should be the same as before the subroutine was called.
- *Local variables* are called "local" because after the subroutine is done, references to all these local variables are removed (i.e., they cannot be accessed).
- Note that popped and deallocated values are still there, as the stack frame does not "clean up" memory used in the stack, but they are no longer kept track of by `sp` and will be overwritten by other uses of the stack.



Before and after calling a subroutine

Before calling the subroutine:

- Push parameter(s) to the stack

After returning from the subroutine

- Deallocate parameter(s) from the stack

Example 1: Putting it together

- Write a program that calculates the value of f(x) where
  - f(x) = g(x) + h(x),

  where g(x) = x^2 and h(x) = x+0x18
- Use r1 to store g
- Use r2 to store h
- The output is stored in r0
- Need to preserve r1 and r2
- No local variable

```
.global _start
_start:

mov sp, #0
mov fp, #0
ldr r1, =0x08
stmfd sp!, {r1} // push parameter to stack (Point 1)
bl Sub
add sp, sp, #4 // Deallocate the parameter (Point 7)

_stop:
b _stop

Sub:
//Prologue
stmfd sp!, {fp, lr} // Step 1 (Point 2)
mov fp, sp //Step 2 (Point 3)
//Step 3: No local variable
stmfd sp!, {r1-r3} // Step 4: Preserve r1 to r3 (Point 4)

ldr r3, [fp, #8] // Retrieve parameter and store in r3
mul r1, r3, r3   // Use r1 to store g
add r2, r3, #0x18 // Use r2 to store h

add r0, r1, r2 // Compute sum of g and h and save in
               // r0 as the return value.

//Epilogue
ldmfd sp!, {r1-r3} // Step 1 (Point 5)
// Step 2: No local variable
ldmfd sp!, {fp, pc} // Step 3 (Point 6)

.end
```

Pt	Stack	sp	fp
1	fffffffe0 aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa fffffff0 aaaaaaaa aaaaaaaa aaaaaaaa 00000008	FFFFFFFC	0
2	fffffffe0 aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa fffffff0 aaaaaaaa 00000000 00000014 00000008	FFFFFFF4	0
3	fffffffe0 aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa fffffff0 aaaaaaaa 00000000 00000014 00000008	FFFFFFF4	FFFFFFF4
4	fffffffe0 aaaaaaaa aaaaaaaa 00000008 00000000 fffffff0 00000000 00000000 00000014 00000008	FFFFFFE8	FFFFFFF4
5	fffffffe0 aaaaaaaa aaaaaaaa 00000008 00000000 fffffff0 00000000 00000000 00000014 00000008	FFFFFFF4	FFFFFFF4
6	fffffffe0 aaaaaaaa aaaaaaaa 00000008 00000000 fffffff0 00000000 00000000 00000014 00000008 00000000	FFFFFFFC	0
7	fffffffe0 aaaaaaaa aaaaaaaa 00000008 00000000 fffffff0 00000000 00000000 00000014 00000008 00000000 e3a0d000	0	0

Example 2: Nested Subroutine Revisited

- The following program calculates the value of f(0x08) where
  - f(x) = g(x+0x18) + 0x05
  - g(x) = x+0x12
- No local variables in both subroutines
- r0 is used to store the output

```
ldr sp, =0
ldr fp, =0
ldr r1, =0x08
stmfd sp!, {r1} //push parameter of f to
stack (Point 1)
bl f
add sp, sp, #4 //deallocate parameter for f
(Point 14)
Here:
b Here

f:
// Prologue
stmfd sp!, {fp, lr} // Step 1 (Point 2)
mov fp, sp // Step 2 (Point 3)
// Step 3: No local variable
stmfd sp!, {r1} // Step 4 (Point 4)

ldr r1, [fp, #8] // Retrieve parameter of f
add r1, r1, #0x18 // generate parameter for g
stmfd sp!, {r1} // push parameter of g to
stack (Point 5)

bl g

add sp, sp, #4 // deallocate parameter for g
(Point 11)
add r0, r0, #0x05 // Calculate output
parameter and store in r0

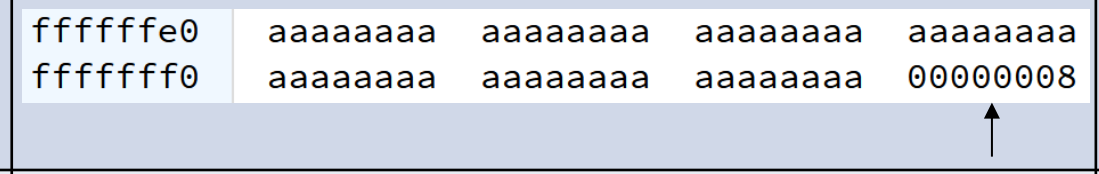
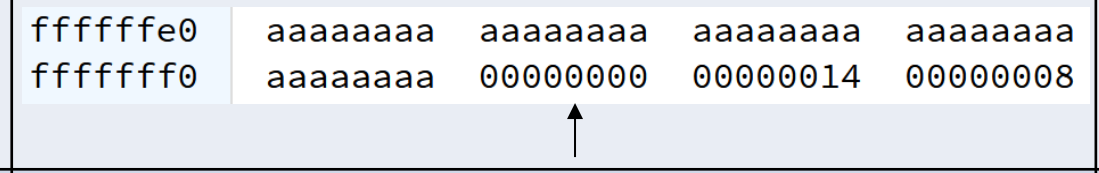
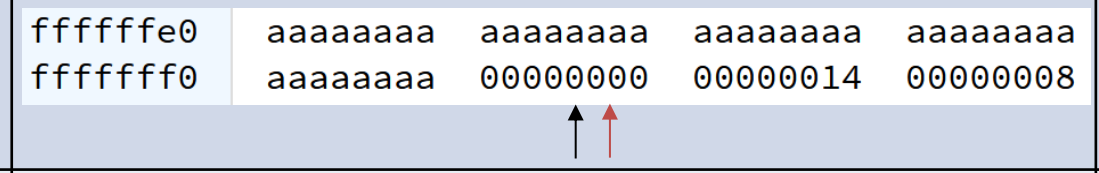
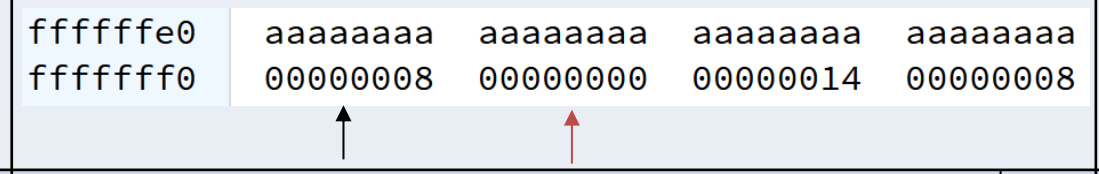
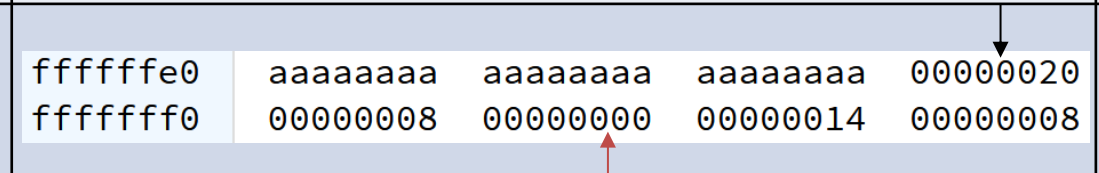
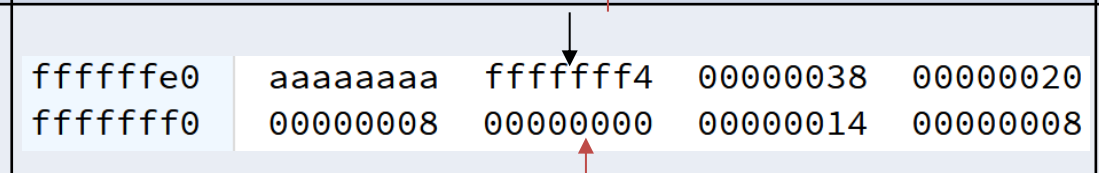
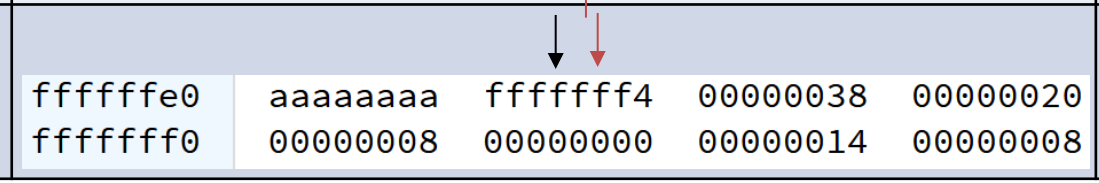
//Epilogue
ldmfd sp!, {r1} // Step 1 (Point 12)
//Step 2: No local variable
ldmfd sp!, {fp, pc} // Step 3 (Point 13)
```

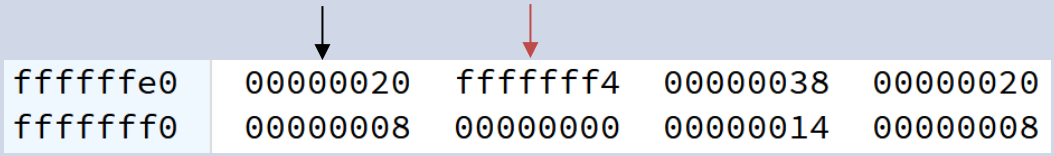
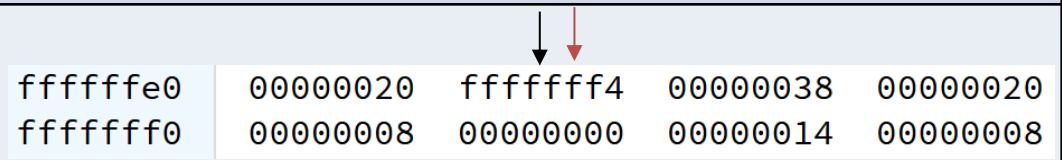
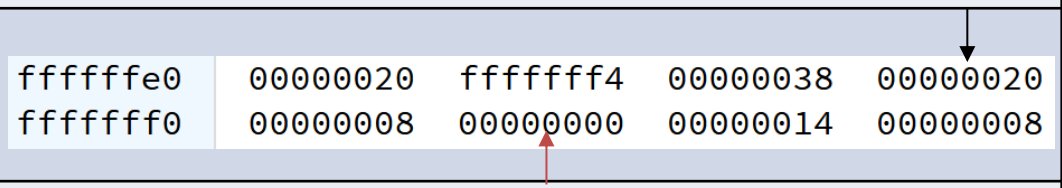
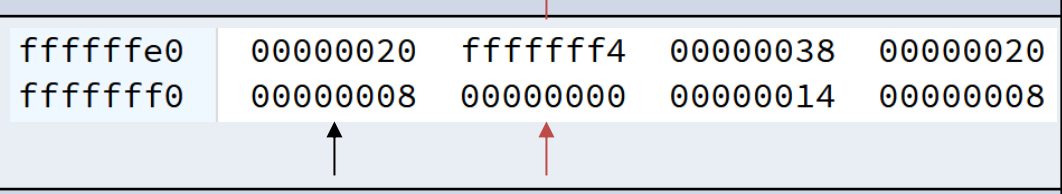
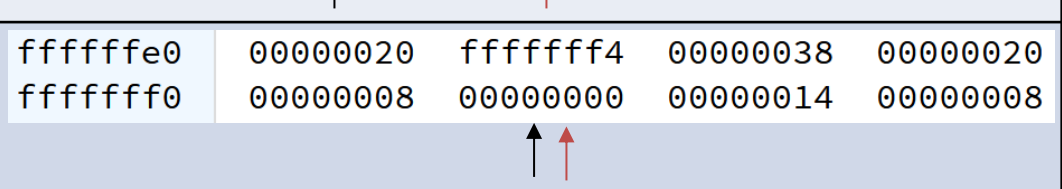
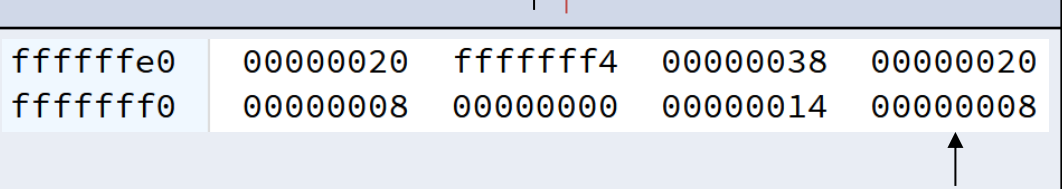
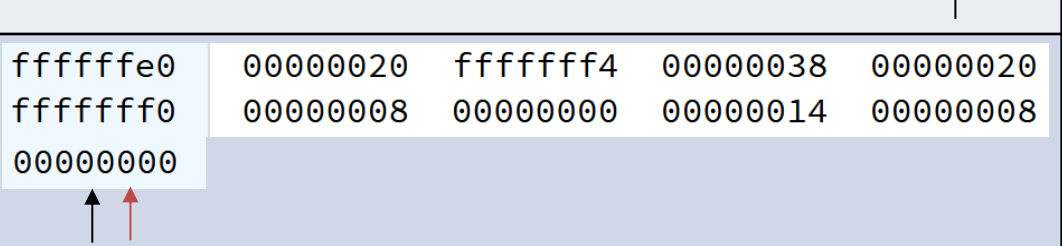
```
g:
// Prologue
stmfd sp!, {fp, lr} // Step 1 (Point 6)
mov fp, sp // Step 2 (Point 7)
stmfd sp!, {r1} (Point 8)

ldr r1, [fp, #8]
add r0, r1, #0x12 // Calculate output
parameter and store in r0

//Epilogue
ldmfd sp!, {r1} // Step 1 (Point 9)
//Step 2: No local variable
ldmfd sp!, {fp, pc} // Step 3 (Point 10)

.end
```

Pt	Stack	sp	fp
1		FFFFFFFC	0
2		FFFFFFF4	0
3		FFFFFFF4	FFFFFFF4
4		FFFFFFF0	FFFFFFF4
5		FFFFFFEC	FFFFFFF4
6		FFFFFFE4	FFFFFFF4
7		FFFFFFE4	FFFFFFE4

Pt	Stack	sp	fp
8		FFFFFFE0	FFFFFFE4
9		FFFFFFE4	FFFFFFE4
10		FFFFFFEC	FFFFFFF4
11		FFFFFFF0	FFFFFFF4
12		FFFFFFF4	FFFFFFF4
13		FFFFFFFC	0
14		0	0

🧠 Micro-Challenge: Review

1. Which register **points to the top of the stack**?

A. fp
B. lr
C. sp
D. pc

🧠 Micro-Challenge: Review

2. What is the main purpose of the **frame pointer (fp)**?

A. Execute instructions
B. Track the top of the stack
C. Provide a stable reference point for parameters and local variables
D. Store the return value

🧠 Micro-Challenge: Prologue Understanding

3. Why do we copy sp into fp during the prologue?

```
mov fp, sp
```

A. To free stack memory
B. To create a fixed reference point for accessing parameters and local variables in the current stack frame
C. To reset the stack
D. To return from the subroutine

🧠 Micro-Challenge: Parameter Access

5. Suppose the parameter x is stored at `[fp, #8]`. What does this mean?

A. x is stored in register r8
B. x is located 8 bytes above fp in the stack frame
C. x is stored in sp
D. x is a local variable

🧠 Micro-Challenge: Parameter Access

6. For the subroutine using a parameter mentioned in Q5, what does the following instruction do?

```
ldr r3, [fp, #8]
```

A. r3 stores the address of fp
B. r3 retrieves a parameter from the stack frame
C. r3 becomes the stack pointer
D. r3 stores the return address

🧠 Micro-Challenge: Parameter Deallocation

7. Consider the following code fragment before calling a subroutine:

```
stmfd sp!, {r1}
bl Sub
add sp, sp, #4
```

What is the purpose of the third instruction?

```
add sp, sp, #4
```

A. To restore the value of register r1
B. To return from the subroutine
C. To deallocate the parameter from the stack after the subroutine call
D. To allocate space for a new parameter

Example 3: Pass by reference (Lab 6)

- Suppose you have the following data:

```
.data
.align
x:
.word 15
y:
.word 9
```

- You want to swap the two values.
- You could pass the values 15 and 9 to the stack and swap their positions in the stack, but that is not what you want to achieve. This “pass-by-value” approach does not work.
- You want to pass the *addresses* of 15 and 9 as parameters to the stack *so that the values in the two memory locations get swapped.* This is referred to as “**pass by reference**”.

Swap: Pass by reference

Main Program

```
16 .global _start
17 _start:
18
19 ldr     r1, =y          // get address of second parameter
20 stmfd   sp!, {r1}       // push second parameter onto stack
21 ldr     r1, =x          // get address of first parameter
22 stmfd   sp!, {r1}       // push first parameter onto stack
23 bl      swap            // call subroutine
24 add     sp, sp, #8      // release parameter memory
25
26 _stop:
27 b       _stop
```

Data Storage

```
70 .data
71 .align
72 x:
73 .word    15
74 y:
75 .word    9
76
77 .end
```

Swap: Pass by reference

---

swaps location of two values in memory.

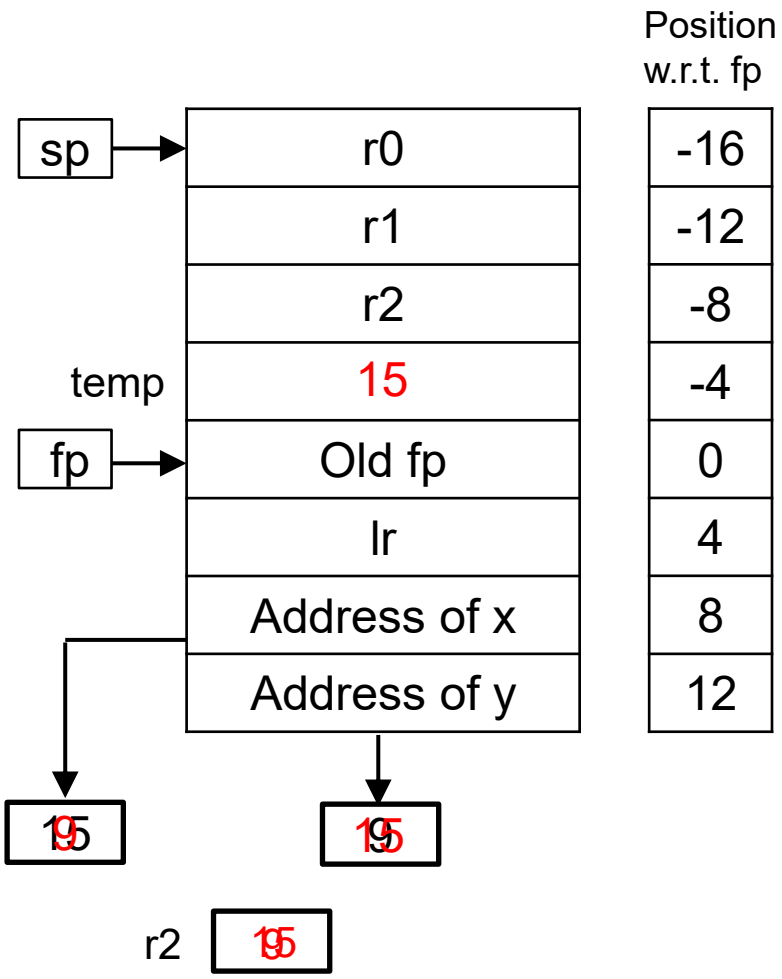
Parameters:
  [fp, #8] – address of x
  [fp, #12] – address of y
Local variable
  temp (4 bytes on stack)
Uses:
  r0 - stores address of x
  r1 - stores address of y
  r2 - registers for swapping operations

```
36 swap:
37 // Prologue
38 stmfd   sp!, {fp, lr}        // push frame pointer
39 mov     fp, sp           // save current stack top to frame pointer
40 sub     sp, sp, #4       // set aside space for local variable temp
41 stmfd   sp!, {r0-r2}     // preserve other registers
42
43 ldr     r0, [fp, #8]     // put address of x in r0
44 ldr     r1, [fp, #12]     // put address of y in r1
45
46 ldr     r2, [r0]         // put value of x in r2
47 str     r2, [fp, #-4]    // copy value of x to temp
48
49 ldr     r2, [r1]         // put value of y in r2
50 str     r2, [r0]         // store value of y in address of x
51
52 ldr     r2, [fp, #-4]    // get temp
53 str     r2, [r1]         // store temp in address of y
54
55 //Epilogue
56 ldmfd   sp!, {r0-r2}     // pop preserved registers
57 add     sp, sp, #4       // remove local storage
58 ldmfd   sp!, {fp, pc}        // pop frame pointer
59
60 .data
61 .align
62 x:
63 .word 15
64 y:
65 .word 9
66
67 .end
```

Swap: Pass by reference

```
36 swap:
37 // Prologue
38 stmfd   sp!, {fp, lr}       // push frame pointer
39 mov     fp, sp             // save current stack top to frame pointer
40 sub     sp, sp, #4         // set aside space for local variable temp
41 stmfd   sp!, {r0-r2}       // preserve other registers
42
43 ldr     r0, [fp, #8]       // put address of x in r0
44 ldr     r1, [fp, #12]      // put address of y in r1
45
46 ldr     r2, [r0]           // put value of x in r2
47 str     r2, [fp, #-4]      // copy value of x to temp
48
49 ldr     r2, [r1]           // put value of y in r2
50 str     r2, [r0]           // store value of y in address of x
51
52 ldr     r2, [fp, #-4]      // get temp
53 str     r2, [r1]           // store temp in address of y
54
55 //Epilogue
56 ldmfd   sp!, {r0-r2}       // pop preserved registers
57 add     sp, sp, #4         // remove local storage
58 ldmfd   sp!, {fp, pc}       // pop frame pointer
59
60 .data
61 .align
62 x:
63 .word 15
64 y:
65 .word 9
66
67 .end
```

Stack after prologue of swap



MinMax subroutine in Lab 6

Finds the minimum and maximum values in a list.

Equivalent of: void MinMax (*start, *end, *min, *max)

Passes addresses of list, end of list, max, and min as parameters.

Parameters:

- start - start address of list (fp + 4)
- end - end address of list (fp + 8)
- min - address of minimum result (fp + 12)
- max - address of maximum result (fp + 16)

Uses:

- r0 - address of start of list
- r1 - address of end of list
- r2 - minimum value so far
- r3 - maximum value so far
- r4 - address of value to process

```
61  MinMax:
62  stmfd   sp!, {fp}       // preserve frame pointer
63  mov     fp, sp          // Save current stack top to frame pointer
64  // allocate local storage (none)
65  stmfd   sp!, {r0-r4}    // preserve other registers
66
67  ldr     r0, [fp, #4]    // Get address of start of list
68  ldr     r2, [r0]        // store first value as minimum
69  ldr     r3, [r0], #4    // store first value as maximum
70  ldr     r1, [fp, #8]    // get address of end of list
71
72  MinMaxLoop:
73  cmp     r0, r1          // Compare addresses
74  beq     _MinMax
75  ldr     r4, [r0], #4
76  cmp     r4, r2
77  movlt   r2, r4
78  cmp     r4, r3
79  movgt   r3, r4
80  b       MinMaxLoop
81
82  _MinMax:
83  // Store results to address parameters
84  ldr     r0, [fp, #12]
85  str     r2, [r0]
86  ldr     r0, [fp, #16]
87  str     r3, [r0]
88
89  ldmfd   sp!, {r0-r4}    // pop preserved registers
90  // deallocate local storage (none was allocated)
91  ldmfd   sp!, {fp}       // pop frame pointer
92  bx      lr              // return from subroutine
```

```
98  .data  // Data section
99  .align
100 Data:
101 .word    4,5,-9,0,3,0,8,-7,12    // The list of data
102 _Data:    // End of list address
103 Min:
104 .space 4
105 Max:
106 .space 4
107
108 .end
```

Example 4: Recursion (Lab 7)

- The following C code implements the Eucildean algorithm in computing the Greatest Common Denominator (gcd) of two integers.

```
1  int gcd(a, b) {
2      if(a == b) {
3          return a;
4      } else if(a > b) {
5          return gcd(b, a - b);
6      } else {
7          return gcd(b, a);
8      }
```

Example 4: Recursion (Lab 7)

```
1  int gcd(a, b) {
2      if(a == b) {
3          return a;
4      } else if(a > b) {
5          return gcd(b, a - b);
6      } else {
7          return gcd(b, a);
8      }
```

Principle:

- Condition 1:
  - If a>b, the problem can be reduced using the following property:
    - gcd (a, b) = gcd (b, a-b)
- Condition 2:
  - If b>a, reverse the order by calling gcd (b,a)
  - Inside this gcd call, condition 1 would be satisfied, thereby reducing the problem.
- Eventually, the two input arguments are equal and gcd (a,a) = a

Numerical Examples

- gcd (20, 15)
- gcd (15, 5)
- gcd (5, 10)
- gcd (10, 5)
- gcd (5, 5)

Two arguments are the same.
gcd = 5

- gcd (60, 36)
- gcd (36, 24)
- gcd (24, 12)
- gcd (12, 12)

Two arguments are the same.
gcd = 12

Example 3: Recursion (Lab 7)

Main Program

```
21 ldr    r1, =numbers     // get address of numbers
22 ldr    r2, [r1, #4]     // put second number into register (4 bytes past first number)
23 stmfd  sp!, {r2}        // push second number onto stack
24 ldr    r2, [r1]         // put first number into register
25 stmfd  sp!, {r2}        // push first number onto stack
26 bl     gcd              // call the subroutine
27 add    sp, sp, #8       // release the parameter memory from the stack
28 // Result in r0
29
30 ldr r1, =result
31 str    r0, [r1]    // write result back to memory
```

Data Section

```
82 .data
83 .align
84 numbers:
85 .word 30, 12
86 result:
87 .space 4
88 .end
```

Before calling the subroutine:

- Push second and then first parameter(s) to the stack

After returning from the subroutine

- Deallocate parameter(s) from the stack

gcd subroutine

Finds the Greatest Common Denominator of two integers (recursive)
Uses simple recursive algorithm:

```
int gcd(a, b) {
    if(a == b) {
        return a;
    } else if(a > b) {
        return gcd(b, a - b);
    } else {
        return gcd(b, a);
    }
```

Parameters:
  a - value of first number
  b - value of second number
Returns:
  r0 - GCD of a and b
Uses:
  r0 - value of a - return value
  r1 - value of b

```
58 gcd:
59 stmfd   sp!, {fp, lr}        // preserve frame pointer
60 mov     fp, sp           // save current stack top to frame pointer
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters into registers
65 ldr     r0, [fp, #8]     // get a
66 ldr     r1, [fp, #12]     // get b
67
68 cmp     r0, r1
69 beq     _gcd             // a is the gcd: return in r0
70 subgt   r0, r0, r1       // a > b: gcd(b, a - b)
71                          // else:  gcd(b, a)
72 stmfd   sp!, {r0}        // 2nd parameter - push a onto the stack
73 stmfd   sp!, {r1}        // 1st parameter - push b onto the stack
74 bl      gcd              // recursive call to subroutine
75 add     sp, sp, #8       // release parameter memory from stack
76
77 _gcd:
78 ldmfd   sp!, {r1}   // pop preserved registers
79 // No local variables
80 ldmfd   sp!, {fp, pc}        // pop frame pointer
```

In both cases,
second parameter is r0,
first parameter is r1

Skeleton of the subroutine

```
58 gcd:
59 stmfd   sp!, {fp, lr}      // preserve frame pointer
60 mov     fp, sp             // save current stack top to frame pointer
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters into registers
65 ldr     r0, [fp, #8]      // get a
66 ldr     r1, [fp, #12]      // get b
67
68 cmp     r0, r1
69 beq     _gcd              // a is the gcd: return in r0
70 subgt   r0, r0, r1        // a > b: gcd(b, a - b)
71                           // else:  gcd(b, a)
72 stmfd   sp!, {r0}         // 2nd parameter - push a onto the stack
73 stmfd   sp!, {r1}         // 1st parameter - push b onto the stack
74 bl      gcd               // recursive call to subroutine
75 add     sp, sp, #8        // release parameter memory from stack
76
77 _gcd:
78 ldmfd   sp!, {r1}    // pop preserved registers
79 // No local variables
80 ldmfd   sp!, {fp, pc}       // pop frame pointer
```

Standard Prologue

Just like before and after calling a subroutine in the main program, we need to:

Push parameters to stack before calling subroutine

Deallocate parameters when the subroutine finishes

Standard Epilogue

Example: gcd (18,12)

- gcd (18, 12) from main program
  - gcd (12, 6)
    - gcd (6, 6) → [r0] = 6

Main Program:

```
21 ldr     r1, =numbers
22 ldr     r2, [r1, #4]
23 stmfd   sp!, {r2}
24 ldr     r2, [r1]
25 stmfd   sp!, {r2}
26 bl      gcd   (18,12)
27 add     sp, sp, #8
28 // Result in r0
29
30 ldr r1, =result
31 str     r0, [r1]     //
```

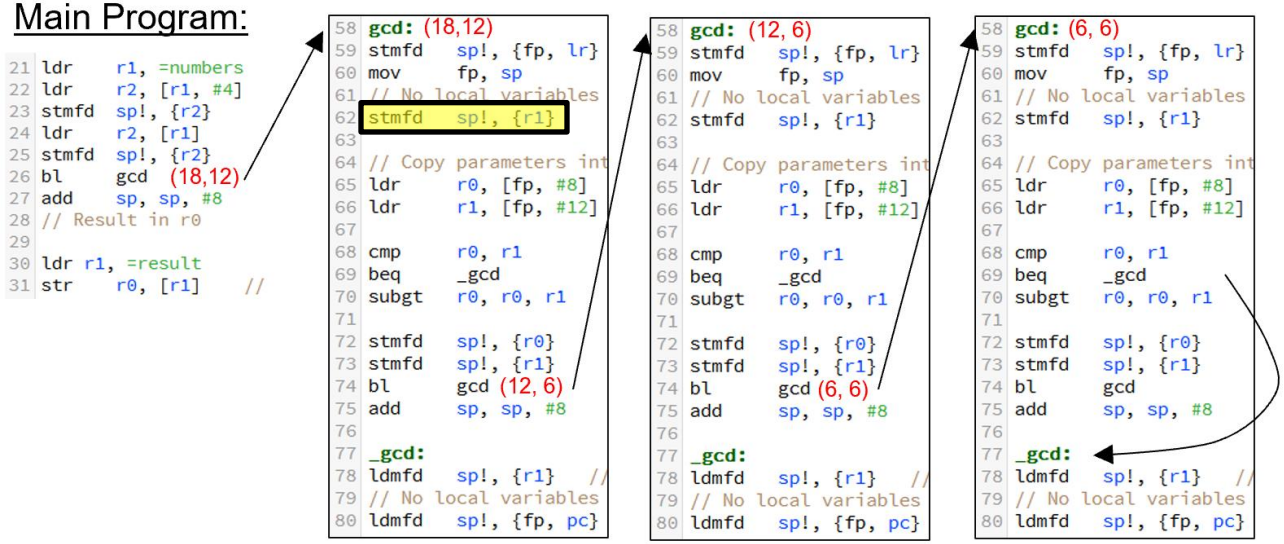
```
58 gcd: (18,12)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (12, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}    //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

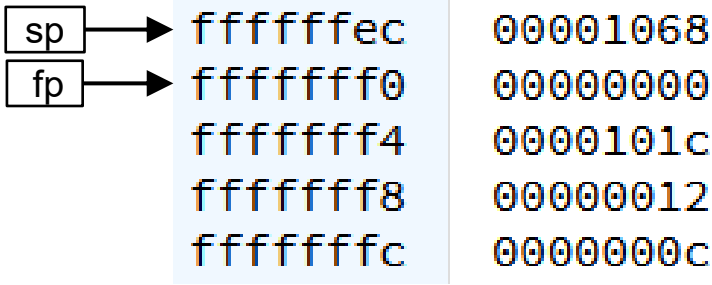
```
58 gcd: (12, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (6, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}    //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

```
58 gcd: (6, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}    //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

Example: gcd (18,12)

1. After execution of prologue for gcd (18, 12):





Activation Record
1st level call
[i.e., gcd (18,12)]

Example: gcd (18,12)

2. After execution of prologue for gcd (12, 6)

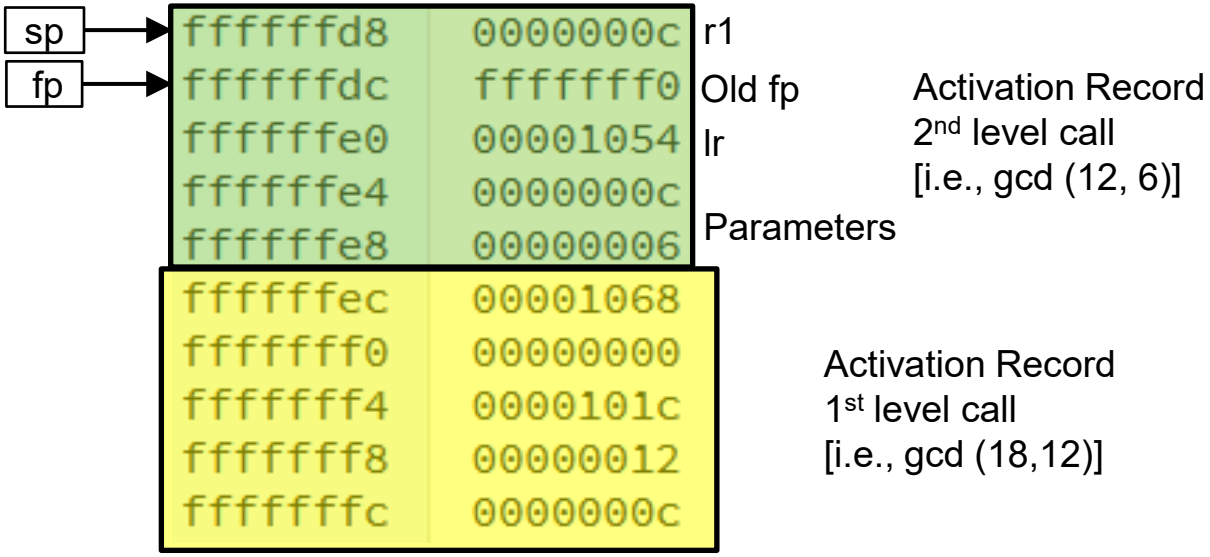
Main Program:

```
21 ldr     r1, =numbers
22 ldr     r2, [r1, #4]
23 stmfd   sp!, {r2}
24 ldr     r2, [r1]
25 stmfd   sp!, {r2}
26 bl      gcd  (18,12)
27 add     sp, sp, #8
28 // Result in r0
29
30 ldr r1, =result
31 str     r0, [r1]     //
```

```
58 gcd: (18,12)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (12, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

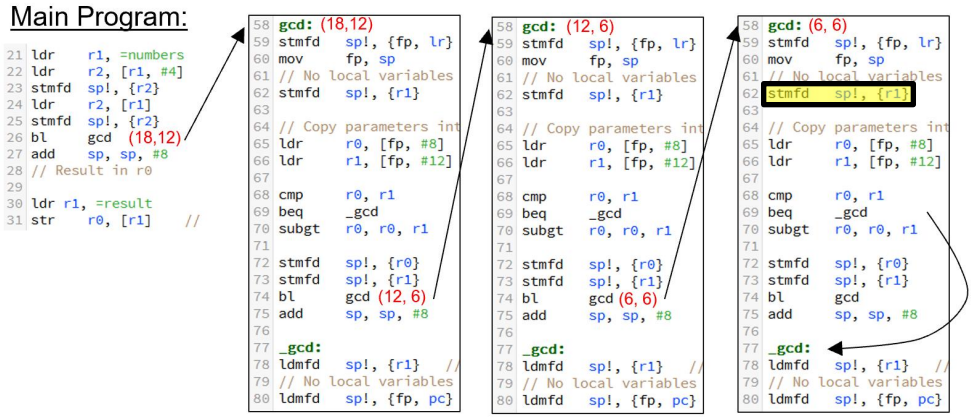
```
58 gcd: (12, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (6, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

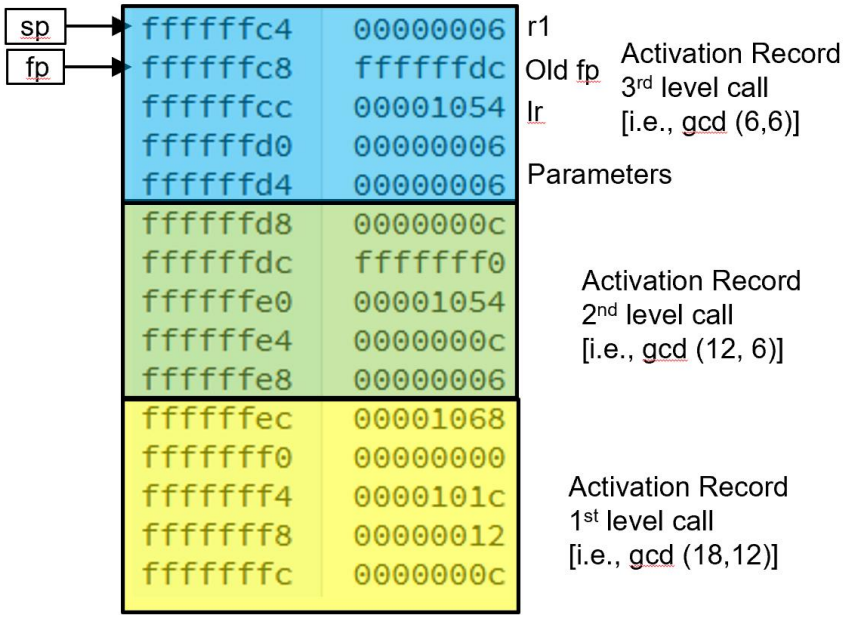
```
58 gcd: (6, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```



Example: gcd (18,12)

3. After the execution of prologue of gcd (6, 6)





Example: gcd (18,12)

4. After execution of epilogue of gcd (6, 6):

Main Program:

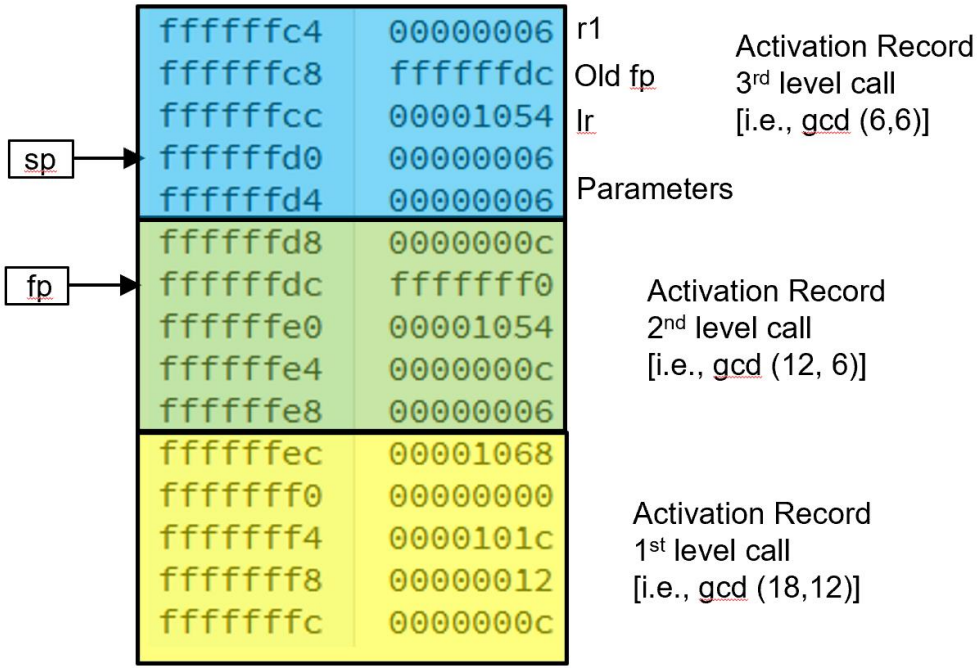
```
21 ldr    r1, =numbers
22 ldr    r2, [r1, #4]
23 stmfd  sp!, {r2}
24 ldr    r2, [r1]
25 stmfd  sp!, {r2}
26 bl     gcd  (18,12)
27 add    sp, sp, #8
28 // Result in r0
29
30 ldr r1, =result
31 str    r0, [r1]    //
```

```
58 gcd: (18,12)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (12, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

```
58 gcd: (12, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (6, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

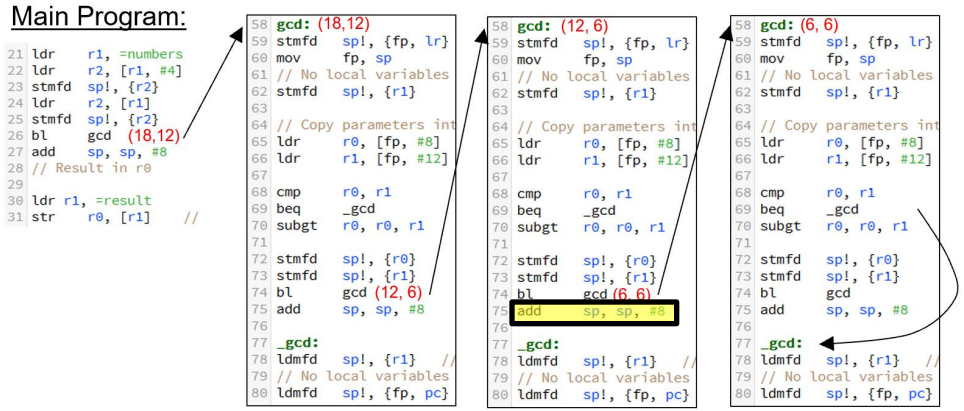
```
58 gcd: (6, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

Result stored at r0
[r0] = 6
[pc] = 00001054

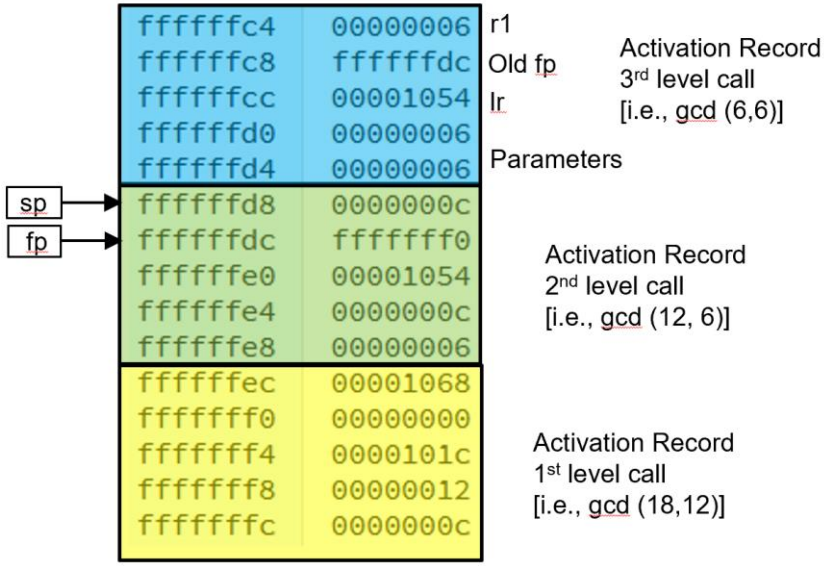


Example: gcd (18,12)

5. After execution of `add sp, sp, #8` after the call gcd (6,6) inside gcd (12, 6).

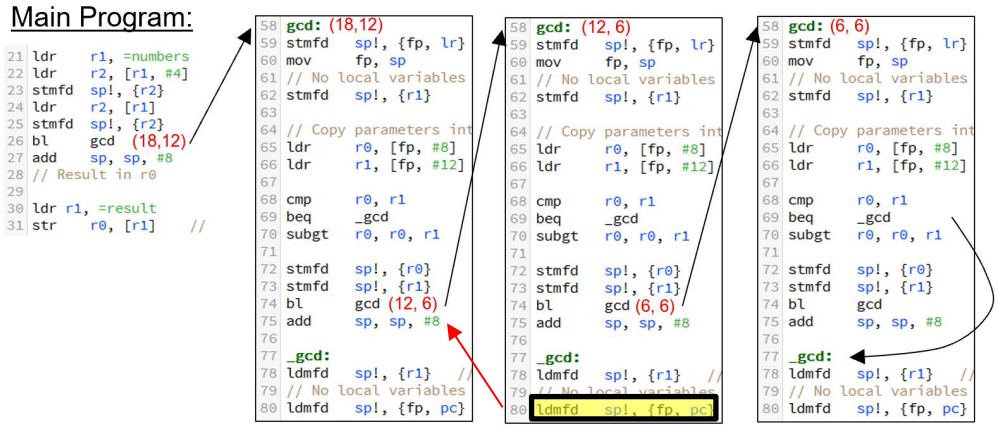


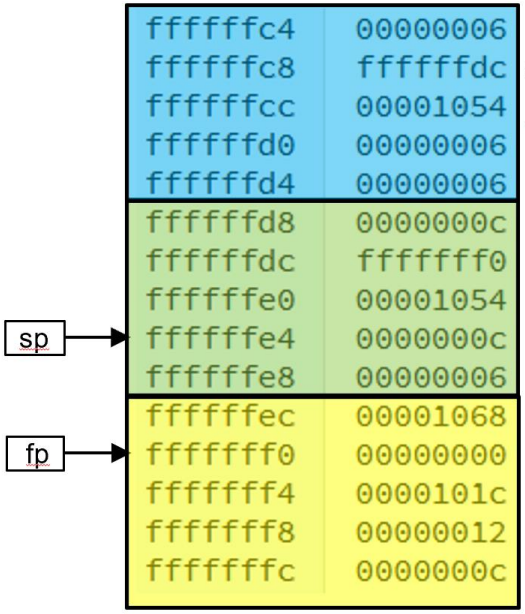
Result stored at r0
[r0] = 6



Example: gcd (18,12)

6. After execution of epilogue of gcd (12,6):





r1
Old fp
lr

Parameters

Activation Record
3rd level call
[i.e., gcd (6,6)]

Activation Record
2nd level call
[i.e., gcd (12, 6)]

Activation Record
1st level call
[i.e., gcd (18,12)]

Result stored at r0
[r0] = 6
[pc] = 00001054

Example: gcd (18,12)

7. After the execution of `add sp, sp, #8` immediately below the call gcd (12, 6) inside gcd (18, 12).

Main Program:

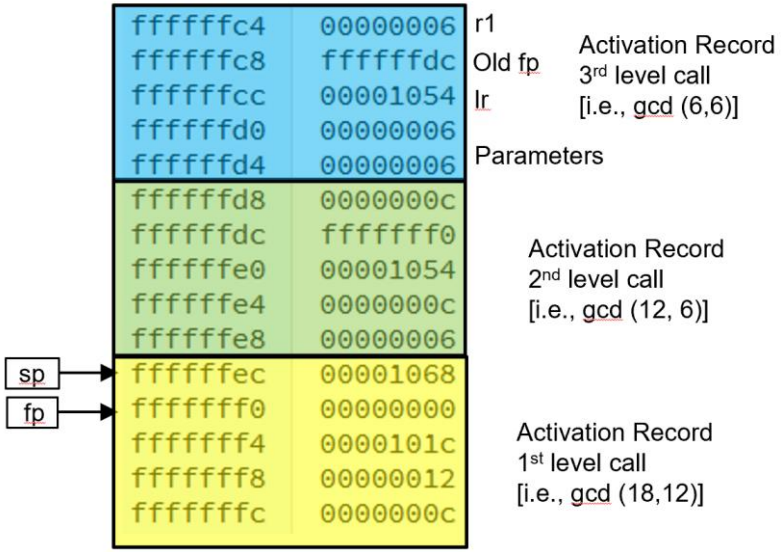
```
21 ldr    r1, =numbers
22 ldr    r2, [r1, #4]
23 stmfd  sp!, {r2}
24 ldr    r2, [r1]
25 stmfd  sp!, {r2}
26 bl     gcd   (18,12)
27 add    sp, sp, #8
28 // Result in r0
29
30 ldr r1, =result
31 str    r0, [r1]    //
```

```
58 gcd: (18,12)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (12, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

```
58 gcd: (12, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd (6, 6)
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

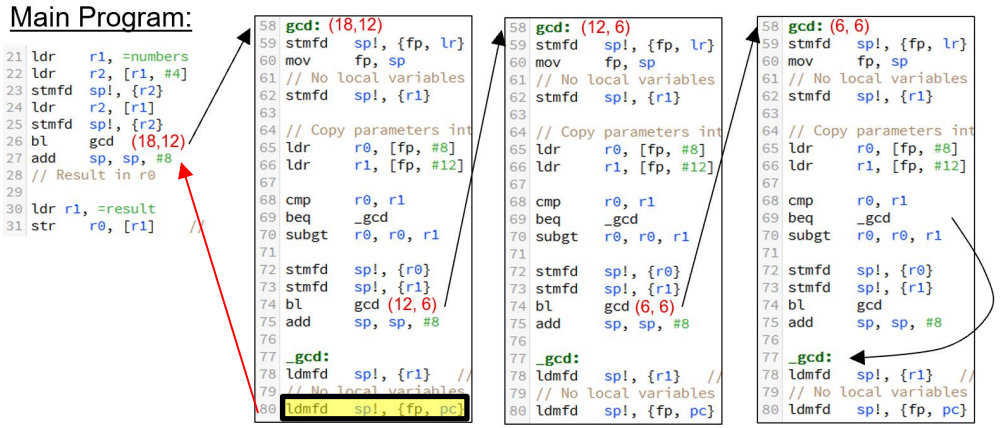
```
58 gcd: (6, 6)
59 stmfd   sp!, {fp, lr}
60 mov     fp, sp
61 // No local variables
62 stmfd   sp!, {r1}
63
64 // Copy parameters int
65 ldr     r0, [fp, #8]
66 ldr     r1, [fp, #12]
67
68 cmp     r0, r1
69 beq     _gcd
70 subgt   r0, r0, r1
71
72 stmfd   sp!, {r0}
73 stmfd   sp!, {r1}
74 bl      gcd
75 add     sp, sp, #8
76
77 _gcd:
78 ldmfd   sp!, {r1}   //
79 // No local variables
80 ldmfd   sp!, {fp, pc}
```

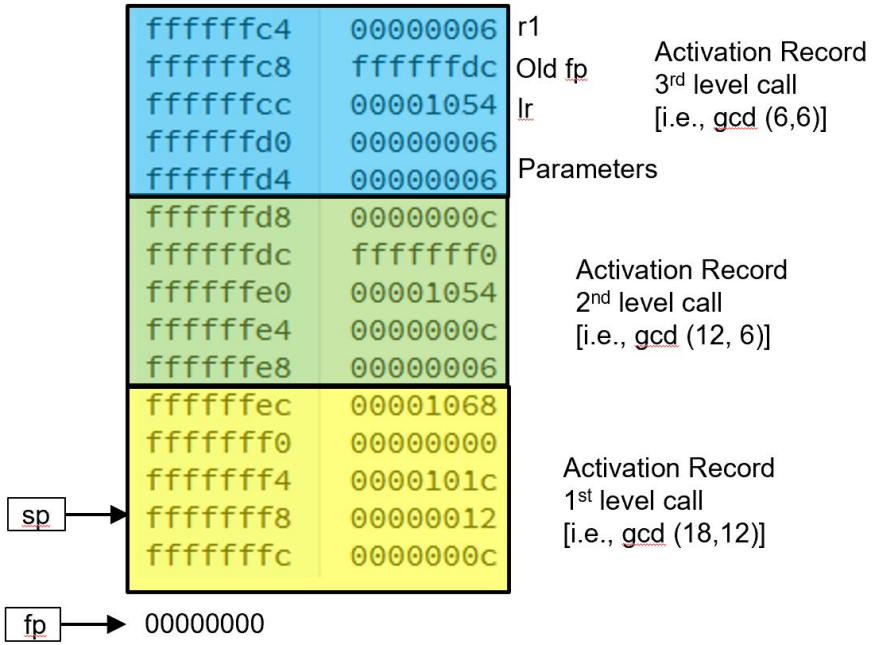
Result stored at r0
[r0] = 6



Example: gcd (18,12)

8. After execution of epilogue of gcd (18, 12):

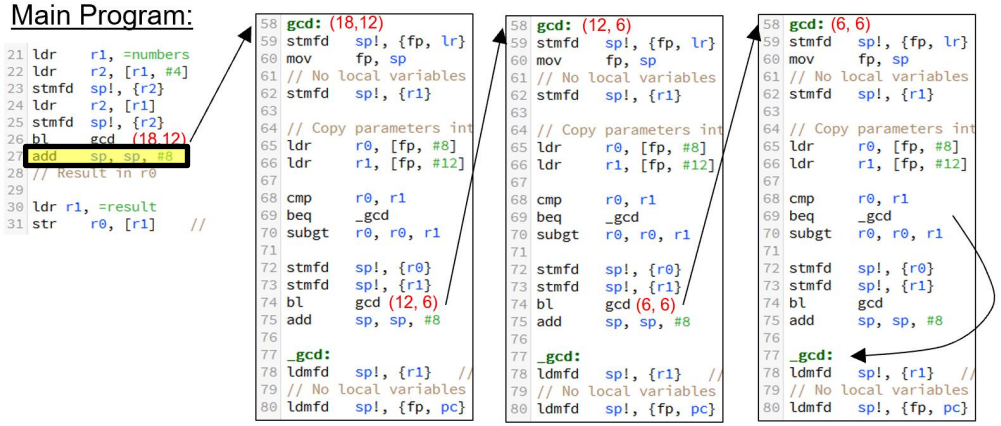




Result stored at r0
[r0] = 6
[pc] = 0000101C

Example: gcd (18,12)

9. After execution of `add sp, sp, #8` at main function:



Result stored at r0
[r0] = 6

